

Aggregation and Joins in Apache Flink

How the framework executes them, what they cost, and where the Kubernetes Operator fits

flink-stream reference environment

11 September 2026

Contents

1	The premise: sharding decides the cost	2
1.1	Two hash functions, two moduli	2
1.2	Making the shuffle visible	3
2	Aggregation	3
2.1	What Flink actually stores	3
2.2	Window assigners and their state profile	3
2.3	Lateness has three tiers	4
2.4	Aggregation in SQL: window table-valued functions	5
2.5	The cost of an updating aggregate	5
2.6	Two-phase aggregation	6
2.7	The low-cardinality ceiling	6
3	Joins	6
3.1	Five kinds, and what each one costs	6
3.2	Regular join	7
3.3	Interval join	7
3.4	Temporal join: the one that makes replays reproducible	8
3.5	Lookup join	8
3.6	Window join	9
3.7	Broadcast: how to avoid a join	9
3.8	What the SQL compiles to	10
4	How Flink executes both	10
4.1	The shuffle	10
4.2	State backend	10
4.3	Watermarks: the clock that drives eviction	11
4.4	Checkpoint barriers	11
4.5	Exactly-once is two-phase commit	11
5	The operator's role	12
5.1	Reconciliation	12

5.2	Application mode versus session mode	12
5.3	Upgrades: what happens to state	13
5.4	Rescaling	13
5.5	The autoscaler	14
5.6	Snapshots as Kubernetes objects	14
5.7	Where the operator does not help	14
6	What the measurements showed	15
7	Failure modes worth meeting once	15
8	Appendix: where to look in the reference environment	15

1 The premise: sharding decides the cost

Aggregation and joins look like SQL concepts. In a stream processor they are mostly a *data movement* problem, and everything expensive about them follows from one fact: the data arrives already split across partitions, and the grouping or join key is usually not the partitioning key.

This document uses a running reference environment to make that concrete. The setup is a 12-partition Kafka topic of payment transactions, three dimension topics, and two Flink jobs – one written in the DataStream API, one in SQL – computing the same kinds of results and writing them to Kafka, Apache Iceberg and YugabyteDB. Every number quoted here was measured on that system while it ran.

1.1 Two hash functions, two moduli

The reason there is no shortcut is that Kafka and Flink shard with different functions over different spaces.

Kafka `partition = (murmur2(keyBytes) & 0x7fffffff) % numPartitions`

Flink `keyGroup = murmurHash(key.hashCode()) % maxParallelism`
 `subtask = keyGroup * parallelism / maxParallelism`

Kafka hashes the serialized *bytes*. Flink hashes the deserialized object’s `hashCode()`, maps it onto a fixed number of **key groups**, and deals contiguous key-group ranges out to subtasks. Different function, different modulus, different space. Even a job that keys by exactly the field Kafka partitioned on has to move almost every record.

There is therefore no co-partitioning optimisation to reach for. **Every keyBy is a real network shuffle**, and a join between two streams keyed differently shuffles both of them.

Key groups also explain why rescaling works at all, which matters later when the operator enters the picture. Parallelism can change; key groups cannot. Restoring a savepoint at a new parallelism is just re-dealing existing key-group ranges to a different number of subtasks. The count, `maxParallelism`, is chosen once and frozen into the state snapshot.

1.2 Making the shuffle visible

In the reference environment each transaction carries `shard_id`, the Kafka partition its key hashes to, computed by the producer itself. Aggregates then count the distinct shards they drew from. The per-minute revenue aggregate, grouped by a `region` attribute that lives in the account dimension rather than in the transaction, reports:

region	txn_count	shards_touched
MIDWEST	458	12
NORTHEAST	461	12
SOUTHEAST	482	12
SOUTHWEST	514	12
WEST	479	12

Five group keys, each assembled from all twelve source shards. That is the shuffle, measured.

2 Aggregation

2.1 What Flink actually stores

The single most important question about any aggregate is *what is in state* – not “the data”, but specifically what the operator needs to answer its next question.

A windowed aggregate can be written two ways, and they differ by orders of magnitude:

```
// buffers every record until the window fires
.window(TumblingEventTimeWindows.of(Duration.ofMinutes(1)))
.process(new MyProcessWindowFunction())

// keeps one accumulator per open window
.window(TumblingEventTimeWindows.of(Duration.ofMinutes(1)))
.aggregate(new TransactionAggregate(), new WindowStatsFunction(...))
```

The second form folds each record into an accumulator on arrival. A window holding a thousand transactions costs one accumulator. The `ProcessWindowFunction` still runs once per window, with access to the key and the window metadata – so nothing is lost except the ability to see individual records, which a correctly-written aggregate does not need.

The equivalent in SQL is automatic: the planner always compiles a `GROUP BY` into an incremental aggregate. The `DataStream` API makes it a choice, which means it can be got wrong.

2.2 Window assigners and their state profile

Assigner	Windows per record	Merging	Notes
Tumbling	1	no	cheapest; boundaries are fixed

Assigner	Windows per record	Merging	Notes
Sliding (hop)	size / slide	no	a 5m/1m window puts every record in 5 panes
Session	1, then merged	yes	windows are created per record and merged
Cumulative	1 per step	no	SQL only; running totals within a max window

Sliding windows are the quiet expense. In the reference job, lab I uses a 5-minute window sliding every minute, so each record is assigned to five panes. It has only five distinct keys, yet it is the busiest operator in the DataStream job – 450 seconds of accumulated busy time against 340 seconds for the CEP operator processing the same input. Low key cardinality did not make it cheap; the fan-out did.

Session windows are the subtle one. They are **merging** windows: each record initially opens its own window, and Flink merges those that overlap. That is why `AggregateFunction.merge` has to be correct. A broken merge produces silently wrong session results while tumbling results stay perfectly fine – a failure mode that survives most test suites.

2.3 Lateness has three tiers

Event-time aggregation has to decide what to do with records that arrive after their window's watermark has passed. There are three outcomes, and only the first two are visible by default:

Timing	Outcome
within <code>maxOutOfOrderness</code>	absorbed; the watermark has not passed the window yet
within <code>allowedLateness</code>	the window fires again , emitting a corrected result
later	dropped, unless routed to a side output

The middle tier deserves attention because it changes the contract with downstream. A window that fires twice emits two records for the same (key, window). Anything consuming that output must be idempotent – which is precisely why the serving tables in the reference environment are upserts keyed on (region, window_start) rather than appends.

The third tier is invisible unless you ask for it. Without `sideOutputLateData` those records vanish and the only evidence is a `numLateRecordsDropped` counter. The reference generator stamps three percent of transactions sixty to a hundred and eighty seconds in the past specifically so this path carries traffic; over the measured run it produced 14,386 dead-letter records that would otherwise have been silently lost.

2.4 Aggregation in SQL: window table-valued functions

```
SELECT region, window_start, window_end, COUNT(*), SUM(amount)
FROM TABLE(
    TUMBLE(TABLE txn_with_account, DESCRIPTOR(event_time), INTERVAL '1' MINUTE))
GROUP BY region, window_start, window_end;
```

Window TVFs replaced the older `GROUP BY TUMBLE(...)` grouped-window syntax, and they are strictly more capable. `window_start` and `window_end` become real columns, which is what makes window Top-N and window joins expressible at all. More importantly for what follows, a window TVF aggregate is **append-only**: it emits once per window and never retracts.

That distinction – append-only versus updating – governs which sinks a query can be written to, and it is the most common planning error in Flink SQL:

Changelog mode	Produced by	Accepted by
append-only	window TVF aggregates, MATCH_RECOGNIZE, interval joins, temporal joins	every sink
updating	regular GROUP BY, Top-N, regular joins	upsert sinks only (upsert-kafka, JDBC with a primary key)

Pointing an updating query at an append-only sink fails at *plan* time with `does not support consuming update changes` – which is a good error to meet once, deliberately, rather than in production.

2.5 The cost of an updating aggregate

Updating aggregates do not merely allow retractions; they *emit* them, and the amplification is visible in the operator metrics. From the running SQL job:

Operator	Records in	Records out
LocalGroupAggregate (pre-shuffle)	132,735	2,184
GlobalGroupAggregate (post-shuffle)	2,184	2,954
Rank (window Top-N)	2,882	9,179

Note the last two rows. The global aggregate emits *more* records than it consumes, because each change to a group produces a retraction of the old value followed by an insertion of the new one. The Top-N operator amplifies further – roughly three times – because a change in ranking retracts every row that moved.

This is not waste; it is the price of correctness for a query whose answer is never final. But it is why a running `GROUP BY` over a high-cardinality key is a materially different proposition from a windowed one, and why `table.exec.state.ttl` exists.

2.6 Two-phase aggregation

The mitigation for shuffle cost is to aggregate *before* the exchange and combine after it. Flink calls this local-global aggregation; it is enabled with:

```
SET 'table.optimizer.agg-phase-strategy' = 'TWO_PHASE';
SET 'table.exec.mini-batch.enabled' = 'true';
SET 'table.exec.mini-batch.allow-latency' = '2 s';
```

Mini-batch is not optional here – local-global aggregation requires it, because the local phase needs a batch boundary at which to emit its partial results.

The effect on the reference job’s region aggregate, measured:

Stage	Records
into the chained temporal join and local aggregate	133,315
across the network exchange (partial aggregates)	449
out of the global aggregate	74

Roughly a **300-fold reduction in shuffle volume**. With `ONE_PHASE`, all 133,315 records would have crossed the network to reach five region keys on two subtasks.

Two-phase aggregation has a prerequisite that is easy to miss: the aggregate function must implement merge. For built-ins this is automatic; for a user-defined aggregate it is a method you have to write, and without it the planner silently falls back to one phase.

2.7 The low-cardinality ceiling

Two-phase aggregation reduces the *volume* crossing the network; it does not increase the number of subtasks that can do the final work. Five region keys means at most five subtasks participate in the global aggregate, whatever the job’s parallelism.

That ceiling is a property of the problem, not a defect to hide. Skew shows up plainly in per-subtask record counts, and the honest responses are to accept it, to add an artificial salt to the key and aggregate twice, or to choose a different grouping. What does *not* work is raising parallelism.

3 Joins

3.1 Five kinds, and what each one costs

Flink offers five stream joins. They differ less in what they compute than in what they keep and whether they can be replayed:

Join	State	Replay-deterministic	Use when
Regular	unbounded – every row of both sides, forever	yes	last resort; always bound with TTL

Join	State	Replay-deterministic	Use when
Interval	bounded by the interval width times the rate	yes	the match is inherently time-local
Temporal	one version per dimension key	yes	a versioned dimension
Lookup	none – queried per row, cached	no	the dimension is authoritative elsewhere
Window	bounded by one window	yes	the comparison is inherently per-window

The regular join is the dangerous one precisely because it is the join anyone would write in a database. It is also the only one whose state grows without bound.

3.2 Regular join

```
SELECT * FROM a JOIN b ON a.k = b.k
```

Flink keeps both sides in keyed state indefinitely, because a row arriving on either side at any future time might match. Nothing in the query bounds that. `table.exec.state.ttl` is the only thing standing between this and a full disk:

```
SET 'table.exec.state.ttl' = '1 h';
```

TTL trades correctness for boundedness, explicitly: a match whose partner arrives after the TTL will not be produced. That is a business decision being made by a configuration setting, which is worth noticing.

3.3 Interval join

```
JOIN auth_events AS au
  ON t.txn_id = au.txn_id
  AND au.event_time BETWEEN t.event_time - INTERVAL '2' SECOND
                        AND t.event_time + INTERVAL '45' SECOND
```

The `BETWEEN` clause is what makes the state bounded. Flink buffers both sides in keyed state and registers cleanup timers; once the watermark proves that no partner can still arrive for a buffered row, that row is evicted. State is therefore proportional to the interval width times the event rate, not to history.

Two properties bite in practice:

It is an inner join. Rows whose partner never arrives simply vanish. In the reference environment the generator deliberately sends about ten percent of authorization events outside the join window and drops eight percent entirely, so the difference is real rather than noise. The interval join consumed 262,196 records across both inputs and emitted 90,354 – and the pipeline sinks the pre-join stream as well, because a transaction without an authorization is still a transaction.

It re-keys. Both topics are partitioned by `account_id`, but the join is on `txn_id`. This is a second complete shuffle, distinct from the one the aggregation performs.

3.4 Temporal join: the one that makes replays reproducible

```
LEFT JOIN accounts FOR SYSTEM_TIME AS OF t.event_time AS a
ON t.account_id = a.account_id
```

A temporal join looks up the version of the dimension that was current *at the fact's own event time*. A plain join would use whatever version happens to be in state when the row is processed, which makes the result depend on when the job ran. Reprocessing last month's data would price it at today's rates.

The right side must be a **versioned table**: a primary key plus a watermark, which in practice means an upsert-kafka source over a compacted topic. Flink keeps a version history per key and emits a left row once the right side's watermark passes that row's timestamp.

3.4.1 The compaction trap

This is where the reference environment produced its most instructive failure. A compacted Kafka topic is a *snapshot*: compaction keeps only the newest record per key. A temporal join needs the opposite – the version that was current when the fact happened. The two requirements are in direct tension, and the default instinct (compact aggressively, the dimension is small) is wrong.

Configured with `min.cleanable.dirty.ratio=0.1` and `segment.ms=60000`, each dimension update deleted the older version that historical facts still needed. The symptom was a steady seventeen percent of aggregate rows resolving to an UNKNOWN region – evenly across every window, so plainly not watermark lag, and not clearing as the job caught up.

The fix is `min.compaction.lag.ms`, which exempts records younger than it from compaction entirely:

```
cleanup.policy=compact
min.compaction.lag.ms=3600000 # an hour of history, always available
min.cleanable.dirty.ratio=0.5 # the default; 0.1 is far too eager
```

Measured before and after on the running cluster: accounts resolving correctly went from 332 of 400 to 397 of 400, and unresolved rows from about seventeen percent to about half a percent. The remainder is the genuine arrival-order case – a fact processed before its dimension record was consumed.

The general form of the rule: **a dimension topic's retained version history must cover how far back the facts replay**. It is a question almost nobody asks until the numbers are wrong.

3.5 Lookup join

```
LEFT JOIN merchant_risk FOR SYSTEM_TIME AS OF t.proc_time AS r
ON t.merchant_id = r.merchant_id
```

The only syntactic difference from a temporal join is `proc_time` instead of `event_time`, and it changes everything. Flink issues a query against the external system per uncached row, with a cache in front:

```
'lookup.cache' = 'PARTIAL',
'lookup.partial-cache.max-rows' = '5000',
'lookup.partial-cache.expire-after-write' = '5min'
```

The trade is stark and worth stating plainly. A lookup join keeps **no Flink state at all** and always sees the freshest value. In exchange it is **not replay-deterministic** – reprocessing last week’s facts produces today’s answers – and it makes the job’s throughput depend on an external database being up and fast.

Neither this nor the temporal join is correct in general. Which one you want depends on whether “correct” means *reproducible* or *current*. The reference environment runs both against the same dimension so the costs can be compared side by side.

3.6 Window join

```
FROM (
    SELECT k, window_start, window_end, COUNT(*) AS n
    FROM TABLE(TUMBLE(TABLE a, DESCRIPTOR(rowtime), INTERVAL '1' MINUTE))
    GROUP BY k, window_start, window_end
) AS x
JOIN (
    SELECT k, window_start, window_end, COUNT(*) AS n
    FROM TABLE(TUMBLE(TABLE b, DESCRIPTOR(rowtime), INTERVAL '1' MINUTE))
    GROUP BY k, window_start, window_end
) AS y
ON x.k = y.k
AND x.window_start = y.window_start
```

Both sides are bucketed into the same windows first, then joined on the window boundaries. Unlike an interval join this can never produce a match straddling a boundary, which makes it the right tool when the comparison is inherently per-window – “did approvals drop in the same minute that spend spiked?”.

3.7 Broadcast: how to avoid a join

The cheapest join is the one that does not happen. When a dimension is small and slow-moving, replicating it to every subtask costs less than shuffling the fact stream:

```
BroadcastStream<Merchant> broadcast = merchants.broadcast(MERCHANT_STATE);
```

```
transactions.keyBy(Transaction::getAccountId)
    .connect(broadcast)
    .process(new MerchantEnrichmentFunction());
```

The fact stream never moves. Flink enforces the invariant that makes this safe: `processBroadcastElement` may write broadcast state, while `processElement` receives a read-only view. Every subtask must converge on identical broadcast state, and if the keyed side could mutate it, subtasks would diverge and restores would be non-deterministic.

Two limits apply. Broadcast state is on-heap and checkpointed once *per subtask*, so the pattern stops paying somewhere in the thousands of dimension rows. And there is no ordering guarantee

between the two inputs – an early fact can arrive before the dimension row that explains it, which is why the reference implementation routes those to a dead-letter topic rather than dropping them.

3.8 What the SQL compiles to

The DataStream and SQL formulations are not different mechanisms. `FOR SYSTEM_TIME AS OF` compiles to essentially the operator that `AccountJoinFunction` in the reference environment writes out by hand: a `KeyedCoProcessFunction` holding the current dimension version per key, buffering facts that arrive early, and expiring both with TTL. Reading the two side by side is the fastest way to stop treating the SQL clause as magic.

The hand-written version also makes the two hard parts explicit, both of which the planner handles silently:

- **Arrival order.** Facts that precede their dimension row are parked and released, not dropped. Dropping them is the most common bug in hand-written enrichment joins, and it only appears at startup or after a rescale.
- **Unbounded state.** The dimension side never expires on its own. TTL bounds it, at the cost of re-reading the compacted topic after a long idle period.

4 How Flink executes both

4.1 The shuffle

`keyBy` inserts a hash exchange. Upstream subtasks partition their output by key group; downstream subtasks read the key-group ranges assigned to them. Flink's job graph makes this visible as a vertex boundary – in the SQL job, `LocalWindowAggregate` and `GlobalWindowAggregate` are separate vertices precisely because the exchange sits between them.

4.2 State backend

Both reference jobs use RocksDB with incremental checkpoints:

```
state.backend.type: rocksdb
state.backend.incremental: "true"
```

The heap backend is faster per access but holds everything in memory. Keyed windows, join buffers and TTL'd maps outgrow that quickly. Incremental checkpointing means only changed SST files are uploaded per checkpoint, which is what keeps a 100 MiB state from costing 100 MiB of upload every thirty seconds.

RocksDB is also what makes state TTL genuinely cheap. Without an explicit cleanup strategy, expired entries are only dropped when read; the compaction filter removes them during normal compaction instead, so state actually shrinks:

```
StateTtlConfig.newBuilder(ttl)
    .setUpdateType(OnCreateAndWrite)
    .cleanupInRocksdbCompactFilter(1_000L)
    .build();
```

4.3 Watermarks: the clock that drives eviction

Every bound discussed so far – window firing, interval-join eviction, temporal-join version selection, CEP pattern timeout – is driven by watermarks, not wall clock. Two settings matter more than the rest.

Out-of-orderness is how far behind the newest timestamp the watermark trails. Too small and correct-but-late records are dropped; too large and every window emits late.

Idleness is the one that silently breaks things. A source subtask reading several shards emits the *minimum* watermark across them, so a single quiet shard freezes event time for the entire job and no window ever fires. On a lightly loaded cluster that is the normal state, not an edge case.

```
WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(5))
                    .withIdleness(Duration.ofSeconds(15));
```

```
SET 'table.exec.source.idle-timeout' = '15 s';
```

A windowed job that produces nothing is almost always this.

4.4 Checkpoint barriers

Checkpoints flow through the job graph as barriers injected at the sources. An operator with several inputs normally waits until the barrier has arrived on all of them – *alignment* – which is exactly where a wide cross-shard exchange stalls.

Unaligned checkpoints let barriers overtake in-flight records, at the cost of persisting that in-flight data as part of the checkpoint. The reference jobs stay aligned until alignment is actually slow:

```
execution.checkpointing.unaligned.enabled: "true"
execution.checkpointing.aligned-checkpoint-timeout: "10s"
```

The measured contrast between the two jobs is instructive:

Job	Vertices	Completed	Avg duration	Max duration	State
DataStream	14	144	871 ms	10.6 s	77 MiB
SQL	26	36	19.1 s	80.4 s	106 MiB

Similar state sizes, twenty times the checkpoint duration. The SQL job has nearly twice the vertices, most of them stateful joins, and it was replaying a backlog with its operators saturated. Checkpoint duration tracks graph width and operator busyness far more than it tracks state size.

4.5 Exactly-once is two-phase commit

End-to-end exactly-once is not an internal property. The Kafka sinks open a transaction per checkpoint interval and commit it when the checkpoint completes. Consumers reading `read_committed` therefore see output lag by up to one checkpoint interval – thirty seconds here. That is the price, not a bug.

Three consequences follow, all of which the reference environment hit:

1. `transaction.timeout.ms` must comfortably exceed the checkpoint interval, or the broker expires the transaction and the job fails on commit.
2. A failed checkpoint is not merely a lost checkpoint; it is output that never becomes visible.
3. Transactional ids are derived as `prefix-subtask-checkpoint`. **Two sinks writing the same table with the same prefix generate identical ids**, and Kafka does what it is designed to do:

```
ProducerFencedException: There is a newer producer with the same
                           transactionalId which fences the current one
```

The job then restart-loops. Two `INSERT` statements into one exactly-once Kafka table is always wrong; either union them into a single statement or give each its own sink table with a distinct prefix.

Not every sink can offer the same guarantee. Flink's JDBC connector supports true exactly-once through XA two-phase commit, but YugabyteDB exposes no `XADataSource`. The reference environment therefore writes at-least-once and makes the *result* idempotent with `INSERT ... ON CONFLICT DO UPDATE` on the natural key. The guarantee depends on the sink system, not on Flink.

5 The operator's role

The Confluent Flink Kubernetes Operator does not execute joins or aggregates. What it owns is the lifecycle around them – and since aggregation and joins are the things that hold state, the lifecycle is where they become operationally interesting.

5.1 Reconciliation

A `FlinkDeployment` is a desired state; the operator works out the stop, snapshot and restart sequence to reach it. Two status fields matter:

Field	Meaning
<code>status.lifecycleState</code>	what the operator is doing: <code>DEPLOYED</code> , <code>STABLE</code> , <code>ROLLING_BACK</code> , <code>FAILED</code>
<code>status.jobStatus.state</code>	what Flink is doing: <code>RUNNING</code> , <code>RESTARTING</code> , <code>FAILED</code>

`STABLE` plus `RUNNING` is the healthy pair.

5.2 Application mode versus session mode

	Application (spec. job present)	Session (no spec. job)
Cluster per job	yes	no, shared
<code>main()</code> runs on	the <code>JobManager</code>	the client
Blast radius of an OOM	that job only	every job on the cluster
Resource accounting	per job	not possible

For stateful pipelines this is not a stylistic choice. A job holding a hundred megabytes of join state should not share a TaskManager with a job that might exhaust the heap. Application mode is the default for anything long-running; session mode exists for interactive SQL work.

5.3 Upgrades: what happens to state

`spec.job.upgradeMode` decides whether the state built up by every join and aggregate survives a change:

Mode	Behaviour	Use when
savepoint	stop with a savepoint, restart from it	the default; state survives a config change
last-state	restart from the last checkpoint without a clean stop	faster, but only safe if the job graph is unchanged
stateless	discard state	the logic changed incompatibly

This is where operator UUIDs earn their keep. A savepoint stores state against an operator's UUID; without an explicit one, Flink derives it from the operator's position in the graph. **Adding an operator anywhere upstream renames every UUID downstream, and the restore silently starts from empty state** – join buffers gone, window accumulators gone, no error. Every stateful operator in the reference job sets one:

```
.process(new AccountJoinFunction(config.stateTtl()))
.name("H: join ref.accounts (versioned, TTL'd)")
.uid("lab-h-account-join");
```

There is an important asymmetry between the two APIs here. A DataStream job's UUIDs are yours to control. A SQL job's operator ids are assigned by the planner from the *plan's structure*, so editing a query renames operators and a savepoint restore either fails or silently drops state. Flink's COMPILED PLAN feature exists to pin a plan across upgrades; until it is in use, a query edit should be treated as a stateless redeploy.

5.4 Rescaling

```
helm upgrade ... --set flink.datastream.parallelism=4
```

With `upgradeMode: savepoint` the operator stops the job with a savepoint and restores it at the new parallelism. This works only because keyed state is stored per key group rather than per subtask – the same mechanism from the opening section. Rescaling re-deals existing key-group ranges.

The constraint that follows: `pipeline.max-parallelism` is the key-group count, it is frozen into the state on first checkpoint, and changing it later invalidates that state. It is chosen once, with headroom, before the first production run.

5.5 The autoscaler

The operator ships an autoscaler that reads source lag and per-vertex busyness and rewrites parallelism **per operator**, not per job. That granularity matters for the subject of this document: in the reference job the sliding-window aggregate is four times busier than the broadcast enrichment feeding it, and scaling the whole job to fix one operator wastes the rest.

Advisory mode computes and logs recommendations without changing anything:

```
job.autoscaler.enabled: "true"
job.autoscaler.scaling.enabled: "false"
```

Two limits are worth knowing. It cannot scale a source usefully past the shard count – twelve shards means twelve useful source subtasks. And it cannot exceed `pipeline.max-parallelism`, which is why that value is set generously up front.

5.6 Snapshots as Kubernetes objects

The operator takes periodic savepoints and exposes on-demand ones as a CRD:

```
kubernetes.operator.periodic.savepoint.interval: "1h"
kubernetes.operator.savepoint.history.max.count: "5"
```

A savepoint differs from a checkpoint in ways that matter specifically for stateful joins and aggregates:

	Checkpoint	Savepoint
Owner	the running job	you
Format	backend-specific, incremental	canonical, self-contained
Restore at different parallelism	no	yes
Restore into a modified graph	no	yes , if UIDs match

5.7 Where the operator does not help

Being clear about the boundary is more useful than overselling it.

The operator does not understand what a job computes. It cannot tell you that a regular join's state is growing without bound, that a temporal join is silently resolving to NULL because compaction ate the dimension history, or that two sinks share a transactional-id prefix. Those are properties of the query, and they are found by looking at operator metrics and at the output – which is why the reference environment carries `shards_touched` and `subtask_index` through to the serving tables.

It also cannot make a startup race safe on its own. In application mode a job's `main()` runs on the JobManager and connects to Kafka, Schema Registry and the sinks immediately; a dependency that is not up throws in `main()`, which is a *deployment* failure rather than a job failure, so Flink's restart strategy does not apply. Both halves of the answer live in the deployment: an init container that waits, and

```
kubernetes.operator.job.restart.failed: "true"
```

6 What the measurements showed

A summary of the numbers quoted above, all from the running reference environment.

Observation	Value
Group keys assembled from all source shards	5 regions, 12 of 12 shards each
Shuffle reduction from two-phase aggregation	133,315 records to 449 partials (~300x)
Updating aggregate amplification	2,184 in, 2,954 out
Window Top-N amplification	2,882 in, 9,179 out (~3.2x)
Interval join selectivity	262,196 in, 90,354 out
MATCH_RECOGNIZE selectivity	132,411 in, 3 matches
Late records captured by side output	14,386
Temporal join resolution before compaction fix	332 / 400 accounts
Temporal join resolution after compaction fix	397 / 400 accounts
Checkpoint duration, DataStream job	871 ms avg (14 vertices)
Checkpoint duration, SQL job	19.1 s avg (26 vertices)

7 Failure modes worth meeting once

Every entry here was hit while building the reference environment, and each is a consequence of something discussed above rather than a configuration typo.

Symptom	Cause
Windowed job produces nothing	one idle shard pinning the watermark; set source idle-timeout
Temporal join returns NULL for everything	dimension has no version at or before the fact's event time
Temporal join returns NULL for a steady fraction	compaction removed the version history; set <code>min.compaction.lag.ms</code>
does not support consuming update changes	an updating query pointed at an append-only sink
ProducerFencedException, job restart-loops	two exactly-once sinks sharing a transactional-id prefix
Session results wrong, tumbling results fine	<code>AggregateFunction.merge</code> is incorrect
State restored empty after a change	an operator UID moved; set them explicitly and never change them
Regular join fills the disk	no <code>table.exec.state.ttl</code>
Output topic looks empty, job healthy	exactly-once sink; records appear on checkpoint commit

8 Appendix: where to look in the reference environment

Concept	File
Incremental aggregation	functions/TransactionAggregate.java, functions/WindowStatsFunction.java
Session-window merge	functions/TransactionAggregate.java (merge)
Cross-shard aggregation	datastream/TransactionPipelineJob.java (labI_crossShardAggregate)
Broadcast enrichment	functions/MerchantEnrichmentFunction.java
Interval join	functions/AuthJoinFunction.java
Hand-written temporal join	functions/AccountJoinFunction.java
Window TVFs, two-phase aggregation, three sinks	files/sql/30-windows.sql
All five SQL joins, side by side	files/sql/40-joins.sql
MATCH_RECOGNIZE	files/sql/50-patterns.sql
Shuffle instrumentation	functions/ShardObserver.java, common/Shards.java
Operator configuration	charts/flink- stream/templates/flink/_flink- helpers.tpl
Compaction settings	charts/flink-stream/templates/init/job- topics.yaml